



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Facultat d'Informàtica de Barcelona



DEEP KERNEL NETWORKS: NEURAL ARCHITECTURES WITH SVM FOUNDATIONS

JOAN ACERO POUSA

Thesis supervisor

LUIS ANTONIO BELANCHE MUÑOZ (Department of Computer Science)

Degree

Master's Degree in Data Science

Master's thesis

Facultat d'Informàtica de Barcelona (FIB)

Universitat Politècnica de Catalunya (UPC) - BarcelonaTech

Abstract

Key Words:

Acknowledgments

Add acknowledgements

Acronyms

ANN Artificial Neural Network. 8, 10, 22–24, 27, 28, 33, 34

PROPOSAL PROPOSAL- NAME TO DEFINE. 33, 36–38

SVC Support Vector Classifier. 6, 11, 12, 14–16, 26

SVM Support Vector Machine. 6, 8, 10, 11, 14, 17–19, 22, 25–27, 29–39

SVR Support Vector Regressor. 19–22

Contents

1	Introduction	8
1.1	Motivation	8
1.2	Objectives	8
1.3	Thesis Structure	9
2	Background	10
2.1	Supervised Learning	10
2.1.1	Classification	10
2.1.2	Regression	10
2.2	Support Vector Machines	11
2.2.1	Linear Support Vector Classification	11
	Hard-margin Support Vector Classifier (SVC)	11
	Soft-margin SVC	14
2.2.2	Non-Linear Classification and the Kernel Trick	17
	The Kernel Trick	17
	Shift-Invariant Kernels	19
	Random Feature Approximations	19
2.2.3	Support Vector Regression	19
2.3	Artificial Neural Networks	22
2.3.1	Architecture	23
2.3.2	Forward Propagation	23
2.3.3	Activation Function	24
2.3.4	Loss Function	24
2.3.5	Backpropagation	25
2.3.6	Support Vector Machines (SVMs) as Single-Layer Networks	25
2.4	Deep Kernel Learning and the Arc-Cosine Kernel	26
3	State Of The Art	28
3.1	The Neural-Kernel Correspondence	28
3.2	Deep Kernel Learning: Networks as Feature Extractors	29

3.3	Deep Architectures of Stacked Kernel Machines	29
3.4	Deep Learning Without Backpropagation	30
3.5	Feature-Learning Kernel Machines	31
3.6	Positioning of This Work	31
4	My Proposal	33
4.1	Architecture	33
4.1.1	Motivation: Avoiding Representation Collapse	33
4.1.2	Layer Structure	33
4.2	The Arc-Cosine Feature Map	34
4.2.1	The Random Feature Map	34
4.2.2	Why Bochner’s Theorem Is Not Needed	35
4.2.3	Theoretical Interpretation and Its Scope	35
4.2.4	Practical Considerations	36
4.3	Learning Algorithm	36
4.4	Prediction	37
4.5	Computational Complexity	37
4.6	Generality of the Architecture	38
5	Experiments	40
6	Conclusion	41

1 Introduction

SVMs are powerful machine learning models with a strong mathematical foundation. They are versatile due to the kernel trick, include inherent regularisation properties [11], and have been shown to be highly effective in both classification and regression tasks [14]. Their optimisation problem is convex, so training is free of the local minima and initialisation sensitivity that affect neural networks.

This strength, however, does not extend to two increasingly common settings. First, kernel SVMs scale poorly: they require the construction of an $n \times n$ kernel matrix, incurring a cost of $O(n^2)$ in memory and between $O(n^2)$ and $O(n^3)$ in time, which becomes prohibitive for large n [8]. Second, SVMs are shallow models: they apply a single fixed feature transformation, and therefore cannot learn the hierarchical, multi-level representations that underlie the success of deep architectures [24]. While kernel approximation techniques address the first limitation in isolation, doing so also introducing depth, and without sacrificing the convexity that makes SVMs attractive, remains an open problem.

1.1 Motivation

This work is motivated by the goal of extending the strong performance of SVMs to precisely those settings in which they are traditionally less effective: large-scale datasets, where exact kernel methods are computationally infeasible, and highly complex data, where a shallow decision function is insufficient.

We aim to address both limitations simultaneously within a single architecture, while retaining the convex, margin-based optimisation that distinguishes SVMs from neural networks.

1.2 Objectives

The main objective of this thesis is to introduce a novel deep architecture that integrates SVMs with the hierarchical feature extraction characteristic of Artificial Neural Networks (ANNs), in order to improve their performance on large-scale and highly complex datasets. More specifically, we seek to:

- Extending the SVM framework to a multi-layer architecture capable of hierarchical feature learning.
- Addressing the scalability limitations inherent to kernel SVMs through an explicit feature-map approximation.
- Developing a training algorithm that avoids backpropagation and preserves the convex optimisation advantages of SVMs.

- Analysing the computational complexity of the proposed method.
- Validating each architectural design decision through an extensive empirical evaluation.
- Benchmarking the proposed method against state-of-the-art alternatives.

1.3 Thesis Structure

The thesis is organised as follows. Section 2.4 provides an overview of the foundational knowledge required to understand the subsequent discussions. Section 3.6 reviews the state-of-the-art literature, establishing the starting point for our investigation. In Section 4.6, we detail the derivation process of the proposed architecture, including its training algorithm and a computational evaluation. Section 5 presents a complete set of experimental work alongside a discussion of the findings. Finally, Section 6 summarises the conclusions of this work and proposes potential paths for future research.

The source code and implementation details to reproduce the experimental results are publicly available at <https://github.com/joanAcero/MultiLayer-MultiSVM>.

2 Background

This chapter presents the theoretical foundations required for the development of the proposed architecture. Section 2.1 outlines the supervised learning framework. Section 2.2 details the mathematical formulation of SVMs, including margin optimisation, duality, the kernel trick, and its approximation through random features. Section 2.3 reviews the architecture and training mechanisms of ANNs. Finally, the chapter connects these methodologies by representing SVMs as single-layer networks and introducing the arc-cosine kernel, which formally links kernel methods to infinite-width neural layers.

2.1 Supervised Learning

Supervised learning is a fundamental setting in machine learning where the algorithm learns from labelled data to generalise predictions for new, unseen data [5]. This section describes the primary supervised learning tasks addressed by the proposed architecture: classification and regression.

2.1.1 Classification

Classification involves assigning a given input to one of several predefined categories or classes. Formally, given a dataset $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, where $\mathbf{x}_i \in R^d$ represents the feature vector of the i -th instance and $y_i \in \{1, 2, \dots, K\}$ is the corresponding class label, the goal is to learn a function $f : R^d \rightarrow \{1, 2, \dots, K\}$ that minimises the discrepancy between the predicted labels $\hat{y}_i = f(\mathbf{x}_i)$ and the actual labels y_i . The process includes a training phase, where the best mapping function f^* is learned, and a prediction phase, where the trained model is used to classify new inputs. The performance is typically evaluated using a loss function $L(y_i, \hat{y}_i)$, which measures the cost of incorrect predictions.

2.1.2 Regression

Regression aims to predict a continuous output value for a given input. Formally, given a dataset $D = \{(x_i, y_i)\}_{i=1}^n$, where $x_i \in R^d$ is the feature vector of the i -th instance and $y_i \in R$ is the continuous output value, the objective is to learn a function $f : R^d \rightarrow R$ that minimises the difference between the predicted values $\hat{y}_i = f(\mathbf{x}_i)$ and the actual values y_i . Like classification, the process involves a training phase to learn the best hypothesis f^* and a prediction phase to estimate continuous values for new data. The effectiveness is assessed using a loss function $L(y_i, \hat{y}_i)$, which quantifies the error of the predictions.

2.2 Support Vector Machines

The theoretical foundations of SVMs can be traced back to the development of statistical learning theory by Vapnik and Chervonenkis [39]. Building upon this framework, Boser, Guyon, and Vapnik formally introduced the optimal margin classifier in [7]. Subsequently, Cortes and Vapnik extended the method to handle non-separable datasets through the introduction of the soft-margin formulation [11], significantly broadening its applicability to real-world problems.

The following sections provide an explanation of the main principles of SVMs, followed by their advanced extensions. The explanation is inspired by [5] and [11].

We first provide the mathematical formulation of the linear SVM optimisation problem for classification. We then extend the explanation to the non-linear case in Section 2.2.2, and subsequently present the adaptation for regression tasks in Section 2.2.3.

2.2.1 Linear Support Vector Classification

SVCs approach the classification problem using the concept of margin, which is described as the smallest distance between the hyperplane and any of the samples. SVCs find the hyperplane that maximises the margin. In a two-class classification problem, the training data set $\mathbf{x}$ contains n input vectors $\mathbf{x}_1, \dots, \mathbf{x}_n$ and their respective labels $y_1, \dots, y_n$, where $y_i \in \{-1, 1\}$. A hyperplane found in a d dimensional feature space is expressed mathematically as:

$$h(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b \quad (1)$$

where $\mathbf{w} \in R^d$ is the weight vector and $b \in R$ is the bias term. During the testing phase, the SVC classifies new observations $\mathbf{x}$ according to $\text{sign}(h(\mathbf{x}))$.

We define the hyperplane that meets $h(\mathbf{x}) = 0$ as follows:

$$H = \{\mathbf{x} \in R^d \mid \mathbf{w}^T \mathbf{x} + b = 0\} \quad (2)$$

Note that samples which are closest to H are known as support vectors and are responsible for defining the margin.

In Figure 1, the dataset is represented with blue circles representing points of one class and red triangles representing points of the other. The optimised hyperplane, denoted as H , is portrayed as a bold line. The margin is indicated as M . Support vectors are uncoloured data points positioned precisely on the dashed lines.

Hard-margin SVC We first describe the initial SVC proposal, which assumes data is perfectly separable.

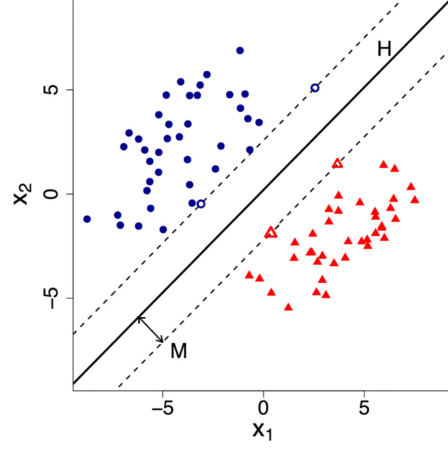


Figure 1: Two-class classification problem in a two-dimensional feature space solved by a hard-margin SVC. Figure taken from [40].

The perpendicular distance from a general point $\mathbf{x}$ to H $h(\mathbf{x})$ defined in Equation 1 is given by (see Figure 4.1 in [5]):

$$\frac{|h(\mathbf{x})|}{\|\mathbf{w}\|} = \frac{|\mathbf{w}^T \mathbf{x} + b|}{\|\mathbf{w}\|} \quad (3)$$

Moreover, since hard-margin SVCs assume the data is linearly separable and focus on solutions without misclassification, we know that a feasible $\mathbf{w}, b$ satisfies $y_i h(\mathbf{x}_i) > 0 \ \forall i$. Thus, the distance from a point $\mathbf{x}_i$ to H is defined as:

$$\frac{y_i h(\mathbf{x}_i)}{\|\mathbf{w}\|} = \frac{y_i (\mathbf{w}^T \mathbf{x}_i + b)}{\|\mathbf{w}\|} \quad (4)$$

Now, suppose we rescale the weight vector $\mathbf{w}$ and the bias b by κ : $\mathbf{w} \rightarrow \kappa \mathbf{w}$ and $b \rightarrow \kappa b$. Note that the distance from $\mathbf{x}_i$ to the decision surface after rescaling remains unchanged:

$$\frac{(\kappa \mathbf{w}^T \mathbf{x}_i + \kappa b)}{\|\kappa \mathbf{w}\|} = \frac{\kappa (\mathbf{w}^T \mathbf{x}_i + b)}{\|\kappa \mathbf{w}\|} = \frac{\kappa (\mathbf{w}^T \mathbf{x}_i + b)}{\kappa \|\mathbf{w}\|} = \frac{\mathbf{w}^T \mathbf{x}_i + b}{\|\mathbf{w}\|} \quad (5)$$

Therefore, we can rescale the margin boundaries to exactly distance 1 for the support vectors:

$$y_i (\mathbf{w}^T \mathbf{x}_i + b) = 1 \quad \forall \mathbf{x}_i \in B \quad (6)$$

where B is the set of support vectors. Since data points of each class must be on or beyond the margin defined by the support vectors, all data points satisfy:

$$y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1 \quad i = 1, \dots, n \quad (7)$$

As previously stated, SVCs seek the specific hyperplane that maximises the margin

between classes. Using Equations 6 and 4, we can derive the mathematical expression of the margin (often referred to as the half-margin, representing the distance from the hyperplane to the support vectors):

$$M = \frac{1}{\|\mathbf{w}\|} \quad (8)$$

To maximise the margin, which is equivalent to minimising $\|\mathbf{w}\|$, we can define the optimisation problem as follows:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad (9)$$

$$\text{subject to } y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 \quad \forall i \quad (10)$$

Here, the factor $\frac{1}{2}$ and the squared term are included for mathematical convenience, simplifying the derivative during optimisation. This expression is known as the primal problem.

To solve the primal problem, we use the method of Lagrange multipliers [23]. We construct the Lagrangian function as:

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^n \alpha_i [y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1] \quad (11)$$

where $\alpha_i \geq 0$ are the Lagrange multipliers and n is the number of data points.

To find the optimal solution, we take the partial derivatives of $\mathcal{L}$ with respect to $\mathbf{w}$ and b and set them to zero:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \mathbf{w} - \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i = 0 \quad (12)$$

$$\frac{\partial \mathcal{L}}{\partial b} = \sum_{i=1}^n \alpha_i y_i = 0 \quad (13)$$

From these conditions, we get:

$$\mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \quad (14)$$

$$\sum_{i=1}^n \alpha_i y_i = 0 \quad (15)$$

Substituting $\mathbf{w}$ back into the Lagrangian, we obtain the dual form:

$$\mathcal{L}_D(\boldsymbol{\alpha}) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \quad (16)$$

Hence, the dual optimisation problem is:

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \quad (17)$$

$$\text{subject to } \sum_{i=1}^n \alpha_i y_i = 0 \quad \text{and} \quad \alpha_i \geq 0 \quad \forall i \quad (18)$$

After optimising the model, the SVC classifies new data points evaluating $\text{sign}(h(\mathbf{x}))$, defined in Equation 1. The hyperplane function can also be expressed as (using equation 14) :

$$h(\mathbf{x}) = \sum_{i=1}^n \alpha_i y_i \mathbf{x}^T \mathbf{x}_i + b \quad (19)$$

Note that this constrained optimisation problem satisfies the Karush-Kuhn-Tucker conditions[21], which require that the following constraints hold for $i = 1, \dots, n$:

$$\alpha_i \geq 0 \quad (20)$$

$$y_i h(\mathbf{x}_i) - 1 \geq 0 \quad (21)$$

$$\alpha_i (y_i h(\mathbf{x}_i) - 1) = 0 \quad (22)$$

From these constraints, we extract that for every data point, either $\alpha_i = 0$ or $y_i h(\mathbf{x}_i) = 1$. Thus, only support vectors have $\alpha_i > 0$, since they satisfy $y_i h(\mathbf{x}_i) = 1$. Therefore, the decision function of the SVC (equation 19) can be redefined for doing the summation over support vectors instead of all data points. This is a crucial characteristic regarding the practical applicability of SVMs since when the model is trained, a significant proportion of the data points can be discarded.

Soft-margin SVC While the hard-margin SVC formulation assumes perfect data separability, this is often incorrect in real-world scenarios. Noise, outliers, or simply intrinsic overlap between classes can cause the hard margin approach to be impractical. Soft-margin SVCs solve this problem by allowing for some misclassification, thereby increasing the model's robustness and generalisation capability.

In soft-margin SVCs, the optimisation objective balances two competing goals: maximising the margin between classes and minimising the classification error. Moreover, this approach introduces an inherent form of regularisation in its formulation, making SVMs less prone to overfitting.

See Figure 2 for a visual reference of the soft-margin SVC. Data points are represented by blue circles and red triangles, one representing each class. One can see that data points are allowed to be misclassified or inside the margin area.

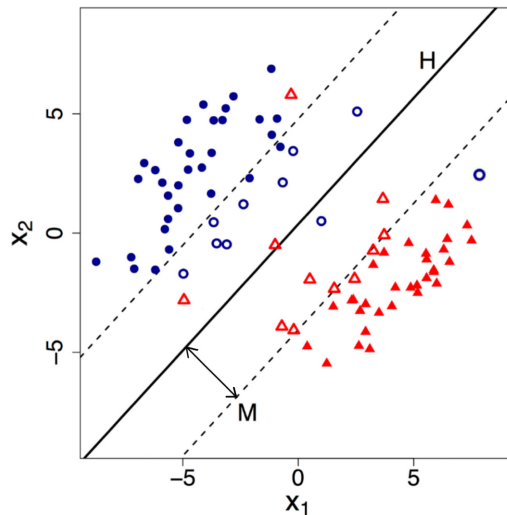


Figure 2: Two-class classification problem in a two-dimensional feature space solved by a soft-margin SVC. Figure taken from [40].

The mentioned capabilities are achieved by introducing slack variables. Slack variables, denoted by $\xi_i \geq 0$, represent the deviation of data points from being correctly classified according to the decision boundary. Each slack variable ξ_i corresponds to a specific data point $\mathbf{x}_i$. If $\xi_i = 0$, it indicates that the point $\mathbf{x}_i$ is correctly classified and lies on the correct side of the margin boundary or over H . If $\xi_i > 0$, it symbolises that the point $\mathbf{x}_i$ is misclassified or lies on the wrong side of the margin boundary, with a magnitude indicating the extent of misclassification.

The optimisation problem is then modified to minimise both the margin size and the sum of these slack variables:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \quad (23)$$

$$\text{subject to } y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i \quad \forall i \quad (24)$$

$$\xi_i \geq 0 \quad \forall i \quad (25)$$

Here, $C > 0$ is a hyperparameter that controls the trade-off between maximising the margin and minimising the classification error. A more significant value of C implies a higher penalty for misclassification, leading to a tighter decision boundary.

To solve this optimisation problem, we again employ the Lagrange multipliers

method [23]. The Lagrangian function for the soft-margin SVC is given by:

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\mu}) = \frac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i - \sum_{i=1}^n \alpha_i [y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 + \xi_i] - \sum_{i=1}^n \mu_i \xi_i \quad (26)$$

Where $\boldsymbol{\alpha} \geq 0$ and $\boldsymbol{\mu} \geq 0$ are the Lagrange multipliers associated with the inequality and slackness constraints, respectively.

We then take the partial derivatives of $\mathcal{L}$ with respect to $\mathbf{w}$, b , and $\boldsymbol{\xi}$, and set them to zero to find the optimal solution:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \mathbf{w} - \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i = 0 \Rightarrow \mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \quad (27)$$

$$\frac{\partial \mathcal{L}}{\partial b} = - \sum_{i=1}^n \alpha_i y_i = 0 \quad (28)$$

$$\frac{\partial \mathcal{L}}{\partial \xi_i} = C - \alpha_i - \mu_i = 0 \Rightarrow \alpha_i = C - \mu_i \quad (29)$$

Substituting these results back into the Lagrangian, we obtain the dual problem:

$$\mathcal{L}_D(\boldsymbol{\alpha}) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \quad (30)$$

subject to the constraints:

$$0 \leq \alpha_i \leq C, \quad (31)$$

$$\sum_{i=1}^n \alpha_i y_i = 0 \quad (32)$$

Once the dual problem is solved, the optimal hyperplane parameters $\mathbf{w}$ and b , along with the slack variables ξ_i can be obtained. To classify a new data point $\mathbf{x}$, the SVC uses $\text{sign}(h(\mathbf{x}))$, defined in 19.

As stated, the soft-margin SVC aims to balance maximising the margin and minimising classification errors. The hinge loss is used to achieve this balance in the training phase. The hinge loss function is defined as:

$$L(y_i, h(\mathbf{x}_i)) = \max(0, 1 - y_i h(\mathbf{x}_i)) \quad (33)$$

The hinge loss penalises misclassified points and points within the margin, with ξ_i representing the amount by which the constraint $y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1$ is violated.

2.2.2 Non-Linear Classification and the Kernel Trick

Many real-world problems involve datasets that exhibit non-linear relationships, rendering linear SVMs insufficient for practical prediction. This limitation underscores the need for non-linear SVMs.

For managing non-linearly tractable data, SVMs can explicitly map the input data into a higher-dimensional space using a function [7]:

$$\phi : R^d \rightarrow R^m \quad \text{with} \quad m \geq d$$

where the ϕ function is generally defined to map data from the original space to a (possibly infinite-dimensional) Hilbert space [46], so that m may be infinite. This mapping allows the SVM to construct a linear decision boundary in the higher-dimensional space, corresponding to a non-linear boundary in the original space (see Figure 3). Thus, the SVM dual optimisation problem (see Equation 30) can be rewritten by replacing the dot product between samples in the original feature space with the inner product of the samples in a high dimensional space. In the classification case, the resulting dual problem is, with the constraints formulation unchanged:

$$\mathcal{L}_D(\boldsymbol{\alpha}) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle \quad (34)$$

The operation $\langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle$ involves mapping the original vectors $\mathbf{x}_i, \mathbf{x}_j$ to the high dimensional space, and posteriorly compute the inner product between them.

Consider the following example illustrating the mapping to a space for a polynomial kernel of degree 2. The steps involved are:

1. Original vectors: $\mathbf{x} = (x_1, x_2), \mathbf{y} = (y_1, y_2)$
2. Mapped vectors: $\phi(\mathbf{x}) = (x_1^2, \sqrt{2}x_1x_2, x_2^2), \phi(\mathbf{y}) = (y_1^2, \sqrt{2}y_1y_2, y_2^2)$
3. Inner product in the higher-dimensional space: $\langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle = x_1^2y_1^2 + 2x_1x_2y_1y_2 + x_2^2y_2^2 = (\mathbf{x}^T \mathbf{y})^2$

The Kernel Trick The kernel trick was introduced in [7]. Using it, inner products in high-dimensional feature spaces can be efficiently computed, avoiding the computational disadvantage of explicit mapping.

Inner products, known as dot products in the context of Euclidean space, are a fundamental operation in linear algebra. For two vectors $\mathbf{u}$ and $\mathbf{v}$ in a vector space, the inner product $\langle \mathbf{u}, \mathbf{v} \rangle$ is a scalar value [17].

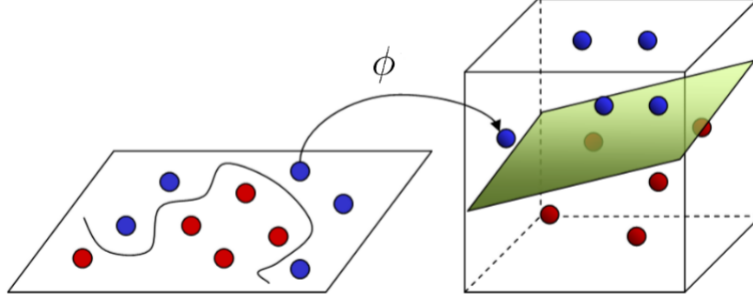


Figure 3: Mapping ϕ from a two-dimensional to a three-dimensional feature space, where data is linearly separable. Image taken from [18].

The main idea is to use a kernel function $K(\mathbf{x}_i, \mathbf{x}_j)$ to compute the inner product in the transformed space instead of explicitly computing $\langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle$:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle \quad (35)$$

By substituting the inner product with a kernel function, the SVM optimisation problem is again rewritten. In the classification case, the resulting dual problem is, with the constraints unchanged:

$$\mathcal{L}_D(\boldsymbol{\alpha}) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j) \quad (36)$$

We now give the definition of the Kernel functions that we use in the posterior Chapters. Each of these functions implicitly computes inner products in different high-dimensional spaces:

- **Polynomial Kernel:** $K(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^T \mathbf{x}_j + c)^d$, where c is a constant and d is the degree of the polynomial.
- **Radial Basis Function (RBF) Kernel:** $K(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma^2}\right)$, where σ defines the width of the Gaussian function. It is powerful for data with complex relationships, creating flexible decision boundaries.
- **Laplacian Kernel:** $K(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|}{\sigma}\right)$, where σ controls the kernel width. It is similar to the RBF kernel but uses the L_1 distance.
- **Cauchy Kernel:** $K(\mathbf{x}_i, \mathbf{x}_j) = \frac{1}{1 + \frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma^2}}$, where σ influences the kernel's width.

By selecting an appropriate kernel function, SVMs can effectively capture and model non-linear structures, enabling the fitting of complex data relationships. However, note that using kernel functions requires the computation of a matrix of inner

products between all data points. This kernel matrix has n^2 entries and requires $O(n^2d)$ operations to build and between $O(n^2)$ and $O(n^3)$ to solve, so both its memory and time costs grow at least quadratically with the number of samples n . As already mentioned, this computation becomes infeasible for large-scale problems [8].

Shift-Invariant Kernels Shift-invariant kernels are a particular class of kernel functions where the value of the kernel depends only on the difference between the input vectors $(x_i - x_j)$ [6]. This property is beneficial in scenarios where the location of features in the input space is not as important as their relative positions. Examples of shift-invariant kernels include the RBF, Laplace and Cauchy kernels.

Random Feature Approximations As noted above, the kernel trick avoids constructing the feature map ϕ explicitly, but at the cost of forming the full $n \times n$ kernel matrix, whose quadratic growth in the number of samples n makes it intractable for large-scale problems. Random feature methods [34] address this limitation from the opposite direction: instead of computing inner products implicitly through a kernel, they construct an *explicit*, finite-dimensional feature map $\mathbf{z} : R^d \rightarrow R^P$ such that the inner product in this low-dimensional space approximates the kernel,

$$K(\mathbf{x}_i, \mathbf{x}_j) \approx \mathbf{z}(\mathbf{x}_i)^T \mathbf{z}(\mathbf{x}_j), \quad (37)$$

where P is the number of random features. Once the inputs are mapped through $\mathbf{z}$, a *linear* model can be trained directly on the transformed data, recovering a non-linear decision boundary in the original space while avoiding the kernel matrix entirely. This reduces the cost from the quadratic $O(n^2)$ of the kernel matrix to $O(nP)$, which is linear in the number of samples and therefore suitable for large-scale settings.

For shift-invariant kernels, Bochner’s theorem [6] guarantees that a continuous shift-invariant kernel is positive-definite if and only if it is the Fourier transform of a non-negative measure. Rahimi and Recht [34] exploited this result to approximate such kernels by sampling frequencies from the corresponding measure, yielding the well-known random Fourier features for the RBF kernel. More broadly, whenever a kernel can be expressed as an expectation over a known distribution, an explicit feature map of this kind can be constructed by Monte Carlo sampling from that distribution.

2.2.3 Support Vector Regression

In regression tasks, SVMs can be adapted to predict continuous values rather than discrete class labels. Similar to classification tasks, variations and extensions address different aspects or challenges in regression problems. We present the Support Vector Regressors (SVRs) standard approach.

In this setting, SVRs aim to fit a function $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$ that has at most ϵ

deviation from the actual target y_i for all training data points $\mathbf{x}_i$. This is known as the ϵ -insensitive loss function.

The primal optimisation problem for the regression problem can be formulated as follows:

$$\min_{\mathbf{w}, b, \xi, \xi^*} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n (\xi_i + \xi_i^*) \quad (38)$$

$$\text{subject to} \quad \begin{cases} y_i - (\mathbf{w}^T \mathbf{x}_i + b) \leq \epsilon + \xi_i \\ (\mathbf{w}^T \mathbf{x}_i + b) - y_i \leq \epsilon + \xi_i^* \\ \xi_i, \xi_i^* \geq 0 \end{cases} \quad (39)$$

Here, ξ_i and ξ_i^* are slack variables that represent deviations above and below the ϵ -tube, respectively. The ϵ -tube is a critical concept in SVRs, defining a margin of tolerance within which no penalty is assigned to errors. The parameter C controls the trade-off between the model complexity and the amount to which deviations larger than ϵ are tolerated.

Refer to Figure 4 for a graphical depiction of these introduced concepts. In this visual representation, data points are illustrated as distinctive red squares. The area between the dashed lines represents the ϵ -tube. Data points residing within this ϵ -tube do not incur any penalisation on the model, as they lie within an acceptable deviation range from the predicted function. These points, therefore, do not contribute to the loss function. Conversely, data points outside the ϵ -tube boundaries are penalised proportionally to their distance from the tube. Slack variables quantify the deviation of these points from the expected margin, denoted as ξ_i and ξ_i^* . Specifically, ξ_i represents deviations above the ϵ -tube, while ξ_i^* represents deviations below it.

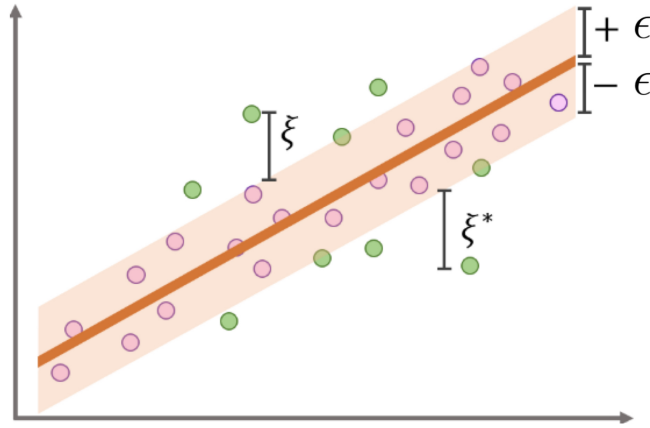


Figure 4: SVRs solving a regression task. Image from [26].

To solve the SVRs primal optimisation problem, we again seek the dual of the problem using Lagrange multipliers [23]. The Lagrangian for the regression problem is given by:

$$\begin{aligned} \mathcal{L}(\mathbf{w}, b, \boldsymbol{\xi}, \boldsymbol{\xi}^*, \boldsymbol{\mu}, \boldsymbol{\mu}^*, \boldsymbol{\alpha}, \boldsymbol{\alpha}^*) = & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n (\xi_i + \xi_i^*) - \sum_{i=1}^n (\mu_i \xi_i + \mu_i^* \xi_i^*) \\ & - \sum_{i=1}^n \alpha_i (\epsilon + \xi_i - y_i + \mathbf{w}^T \mathbf{x}_i + b) - \sum_{i=1}^n \alpha_i^* (\epsilon + \xi_i^* - \mathbf{w}^T \mathbf{x}_i - b + y_i) \end{aligned} \quad (40)$$

where $\boldsymbol{\mu}$, $\boldsymbol{\mu}^*$, $\boldsymbol{\alpha}$, and $\boldsymbol{\alpha}^*$ are the Lagrange multipliers.

Taking the partial derivatives of $\mathcal{L}$ with respect to $\mathbf{w}$, b , ξ_i , and ξ_i^* , and setting them to zero, we get:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \mathbf{w} - \sum_{i=1}^n (\alpha_i - \alpha_i^*) \mathbf{x}_i = 0 \Rightarrow \mathbf{w} = \sum_{i=1}^n (\alpha_i - \alpha_i^*) \mathbf{x}_i \quad (41)$$

$$\frac{\partial \mathcal{L}}{\partial b} = \sum_{i=1}^n (\alpha_i^* - \alpha_i) = 0 \quad (42)$$

$$\frac{\partial \mathcal{L}}{\partial \xi_i} = C - \mu_i - \alpha_i = 0 \Rightarrow \mu_i = C - \alpha_i \quad (43)$$

$$\frac{\partial \mathcal{L}}{\partial \xi_i^*} = C - \mu_i^* - \alpha_i^* = 0 \Rightarrow \mu_i^* = C - \alpha_i^* \quad (44)$$

Substituting these results back into the Lagrangian, we obtain the dual problem:

$$\mathcal{L}_D(\boldsymbol{\alpha}, \boldsymbol{\alpha}^*) = \sum_{i=1}^n y_i (\alpha_i - \alpha_i^*) - \epsilon \sum_{i=1}^n (\alpha_i + \alpha_i^*) - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n (\alpha_i - \alpha_i^*) (\alpha_j - \alpha_j^*) \mathbf{x}_i^T \mathbf{x}_j \quad (45)$$

Subject to the constraints:

$$0 \leq \alpha_i \leq C, \quad (46)$$

$$0 \leq \alpha_i^* \leq C, \quad (47)$$

$$\sum_{i=1}^n (\alpha_i - \alpha_i^*) = 0 \quad (48)$$

Solving the dual problem provides the values of α_i and α_i^* , from which we can determine the weight vector $\mathbf{w}$ and the bias term b . Finally, the decision function for

SVRs can be expressed as:

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b = \sum_{i=1}^n (\alpha_i - \alpha_i^*) \mathbf{x}_i^T \mathbf{x} + b \quad (49)$$

where $\mathbf{w}$ is recovered from the dual variables as $\mathbf{w} = \sum_{i=1}^n (\alpha_i - \alpha_i^*) \mathbf{x}_i$, and b is obtained from the Karush-Kuhn-Tucker conditions on the points lying on the boundary of the ϵ -tube.

As in the classification case, the dual objective and the decision function depend on the inputs only through inner products $\mathbf{x}_i^T \mathbf{x}_j$, so the kernel substitution of Section 2.2.2 applies identically to regression. Replacing each inner product with a kernel function $K(\mathbf{x}_i, \mathbf{x}_j)$ yields the kernelised dual problem:

$$\mathcal{L}_D(\boldsymbol{\alpha}, \boldsymbol{\alpha}^*) = \sum_{i=1}^n y_i (\alpha_i - \alpha_i^*) - \epsilon \sum_{i=1}^n (\alpha_i + \alpha_i^*) - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n (\alpha_i - \alpha_i^*) (\alpha_j - \alpha_j^*) K(\mathbf{x}_i, \mathbf{x}_j) \quad (50)$$

subject to the same constraints, and the corresponding decision function:

$$f(\mathbf{x}) = \sum_{i=1}^n (\alpha_i - \alpha_i^*) K(\mathbf{x}_i, \mathbf{x}) + b. \quad (51)$$

2.3 Artificial Neural Networks

This section presents the essential knowledge of ANNs required for comprehending the thesis contents. Readers are encouraged to consult the provided references for a more detailed exploration of ANNs [16, 5]. In Section 2.3.6, we connect SVMs with ANNs by defining an SVM as a single hidden-layer network.

The conceptualization of ANNs began with the foundational computational model of the neuron proposed by McCulloch and Pitts [29]. Rosenblatt [35] later presented the perceptron, establishing a framework for binary classification using a single layer of neurons capable of learning linear separations. Decades later, Rumelhart, Hinton, and Williams [36] marked a resurgence in ANN research by introducing the backpropagation algorithm for training multi-layer networks. This innovation stimulated the deep learning revolution of the 2000s and 2010s, solidifying ANNs as a fundamental element of modern artificial intelligence [24].

The following sections are based on the books [5, 16].

2.3.1 Architecture

An ANN consists of an input layer, one or more hidden layers, and an output layer. Each neuron in a layer is connected to every neuron in the subsequent layer. See Figure 5 for a visual representation of an ANN with one hidden layer.

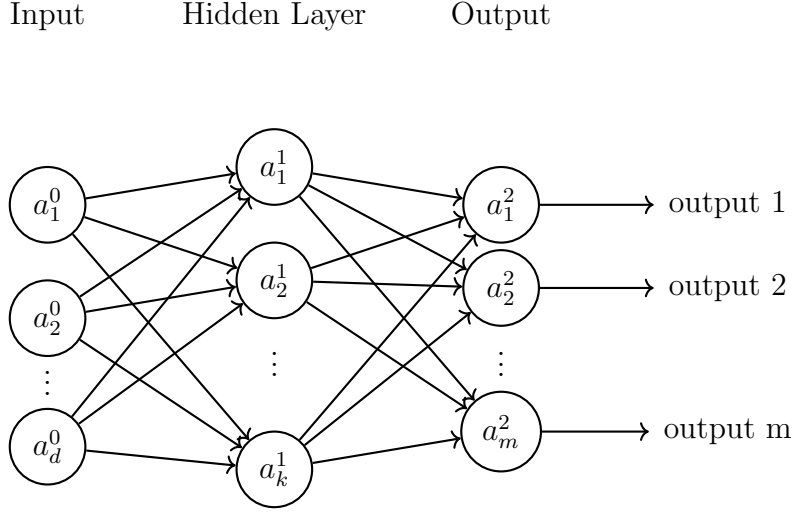


Figure 5: ANN with an input layer of d nodes, a single hidden layer with k nodes, and an output layer of m nodes. Each node is represented as a_b^c , meaning that it is the b -th node of layer number c . Own elaboration.

2.3.2 Forward Propagation

The initial stage in the operation of ANNs is forward propagation [36]. During this process, the network processes input data, transforming it progressively through interconnected layers to produce predictive outcomes. Hence, the forward propagation process computes the neural network output given an input $\mathbf{x}$.

For a neural network with L layers (including the input and output layers), the forward propagation process can be described by the following formula:

$$\mathbf{a}^{(l)} = \sigma(\mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}) \quad \text{for } l = 1, 2, \dots, L \quad (52)$$

Here, $\mathbf{a}^{(0)} = \mathbf{x} \in R^d$ is the input to the network, where d is the number of features of the input sample. $\mathbf{W}^{(l)} \in R^{d_l \times d_{l-1}}$ is the weight matrix of the l -th layer, where d_l is the number of neurons in layer l . $\mathbf{b}^{(l)} \in R^{d_l}$ is the bias vector for the l -th layer, and $\sigma(\cdot)$ represents the activation function applied to the linear transformation of the input.

2.3.3 Activation Function

Activation functions are a key component of artificial neural networks since they introduce non-linearity into the model [37].

Common activation functions include the sigmoid, hyperbolic tangent (tanh), and Rectified Linear Unit (ReLU) functions. The ReLU is one of the most widely used activation functions in deep learning due to its simplicity and effectiveness. It is defined as:

$$\text{ReLU}(x) = \max(0, x) \quad (53)$$

This function outputs the input directly if it is positive; otherwise, it outputs zero.

The ReLU function has several advantages, including computational efficiency and the ability to mitigate the vanishing gradient problem [16].

2.3.4 Loss Function

ANNs also use loss functions to measure the difference between the predicted output $\mathbf{a}^{(L)}$ and the actual target $\mathbf{y}$. Common choices for the loss function include the mean squared error (MSE) or the cross-entropy loss [24].

The MSE loss function is a common choice for regression problems. It calculates the average of the squares of the differences between the predicted values and the actual values. The formula for MSE is:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (54)$$

Where n is the number of samples, y_i is the actual output, and $\hat{y}_i$ is the predicted output.

The cross-entropy loss function is commonly used in classification problems. It measures the dissimilarity between the true distribution and the predicted distribution. The formula for binary cross-entropy (for binary classification) is:

$$\text{Binary Cross-Entropy} = -\frac{1}{n} \sum_{i=1}^n (y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)) \quad (55)$$

Where n is the number of samples, y_i is the actual output (either 0 or 1), and $\hat{y}_i$ is the predicted probability that the i -th sample belongs to class 1.

2.3.5 Backpropagation

Backpropagation was introduced by [36] and is used to update the weights and biases of the neural network to minimise the loss function. It involves computing the gradients of the loss function with respect to the weights and biases using the chain rule and then updating the parameters. The process of the backpropagation algorithm can be summarised as follows:

1. **Forward Pass:** The input data is propagated forward through the network, computing the output of each layer until the final output is obtained. The process is explained in Section 2.3.2.
2. **Loss Computation:** The loss function is computed based on the predicted and actual outputs, as explained in Section 2.3.4.
3. **Backward Pass:** Gradients of the loss function with respect to the weights and biases are computed using the chain rule, starting from the output layer and moving backwards through the network.
4. **Parameter Update:** The weights and biases are updated in the opposite direction of the gradients, using gradient descent or its variants [4].
5. **Iteration:** Steps 1-4 are repeated iteratively for a specified number of epochs or until convergence, gradually reducing the loss and improving the neural network's performance.

2.3.6 SVMs as Single-Layer Networks

Throughout the previous sections, we have consistently pointed out that SVMs are shallow models. This section seeks to justify why SVMs are considered shallow architectures by representing their structure as a network (see Figure 6).

The network representation of an already optimised SVM explains two interesting characteristics of the model: the shallowness of the architecture and the fact that already optimised SVMs only depend on the support vectors of the model. The shallow nature is evident as the network comprises only a single hidden layer. Furthermore, this hidden layer has m nodes, where m is the number of support vectors of the SVM. We note that this corresponds to the dual (kernel) representation, in which each hidden node is a kernel evaluation against a support vector; equivalently, the primal representation casts the same model as a single linear unit acting on the (possibly infinite-dimensional) feature map $\phi(\mathbf{x})$. The architecture proposed in this work generalises the latter, primal feature-map view.

The architecture we propose is based on expanding the shallow model to have more than one hidden layer, thus making it, ideally, more capable of learning intricate patterns in data.

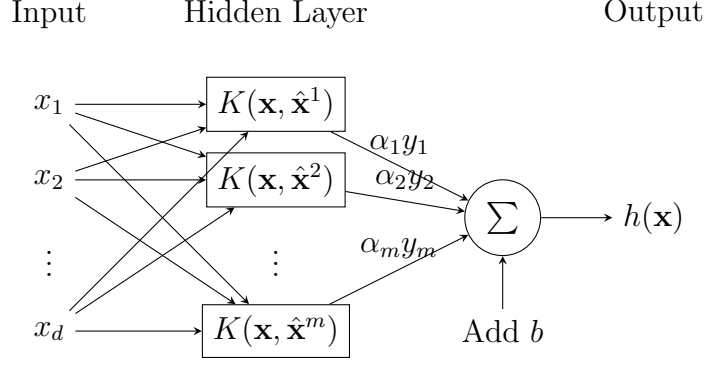


Figure 6: SVC as a single hidden layer network. The input vector is $\mathbf{x} \in R^d$, where d is the number of features. $K(\mathbf{x}, \hat{\mathbf{x}}^i)$ computes the inner product between the input vector and support vector i . Support vectors are $\hat{\mathbf{x}}^i \in R^d$ such that $\alpha_i > 0$, and m is the number of support vectors. Lagrange multipliers of the optimised dual SVM problem are represented as $\alpha_i \forall i = 1 \dots m$. The bias, b , is computed from the support vectors. $y_i \in \{-1, 1\}$ denotes the label of the support vector $\hat{\mathbf{x}}^i$. The decision boundary $h(\mathbf{x})$ evaluates $\text{sign}(\sum_{i=1}^m \alpha_i y_i K(\mathbf{x}, \hat{\mathbf{x}}^i) + b)$. Own elaboration.

2.4 Deep Kernel Learning and the Arc-Cosine Kernel

As established in Section 2.3.6, standard SVMs can be viewed as single-layer neural networks. To bridge the gap between kernel methods and deep learning architectures, Cho and Saul [10] introduced a novel family of positive-definite kernels known as arc-cosine kernels. These kernels are specifically designed to mimic the computation of large neural networks with one or more layers of hidden units. Their positive-definiteness follows directly from their definition as an inner product of feature maps (equivalently, as the expectation in Equation 58 of a product of identical functions of $\mathbf{x}$ and $\mathbf{y}$), rather than from Bochner's theorem; indeed, arc-cosine kernels are not shift-invariant, since they depend on the angle between the inputs rather than on their difference.

The core intuition behind the arc-cosine kernel is that it computes the inner product between the outputs of an infinitely wide single-layer neural network with random Gaussian-distributed weights. To make this precise, consider a single hidden layer with weight vectors drawn independently from a standard Gaussian distribution, $\mathbf{w} \sim \mathcal{N}(0, I)$, where each hidden unit applies the non-linearity

$$g_n(z) = \Theta(z) z^n, \quad (56)$$

with $\Theta(\cdot)$ the Heaviside step function, defined as

$$\Theta(z) = \begin{cases} 1 & \text{if } z > 0, \\ 0 & \text{if } z \leq 0, \end{cases} \quad (57)$$

which acts as a gate that returns 1 when its argument is positive and 0 otherwise. For a given input $\mathbf{x}$, a single hidden unit therefore outputs $g_n(\mathbf{w}^T \mathbf{x}) = \Theta(\mathbf{w}^T \mathbf{x})(\mathbf{w}^T \mathbf{x})^n$: the factor $\Theta(\mathbf{w}^T \mathbf{x})$ ensures the unit activates only when the projection $\mathbf{w}^T \mathbf{x}$ is positive, in which case it outputs $(\mathbf{w}^T \mathbf{x})^n$, and otherwise outputs zero.

If such a layer has a finite number of hidden units, the inner product between the feature representations of two inputs $\mathbf{x}$ and $\mathbf{y}$ is the sum, over all units, of the products of their activations. As the number of hidden units grows to infinity, this sum, once divided by the number of units, converges to its expected value over the weight distribution; the leading factor of 2 in the definition below is the normalising constant that arises from this limit [10]. For two input vectors $\mathbf{x}, \mathbf{y} \in R^d$, the n -th order arc-cosine kernel is precisely this expectation, written in integral form as:

$$k_n(\mathbf{x}, \mathbf{y}) = 2 \int \frac{e^{-\frac{\|\mathbf{w}\|^2}{2}}}{(2\pi)^{d/2}} \Theta(\mathbf{w}^T \mathbf{x}) \Theta(\mathbf{w}^T \mathbf{y}) (\mathbf{w}^T \mathbf{x})^n (\mathbf{w}^T \mathbf{y})^n d\mathbf{w} \quad (58)$$

where the term $\frac{e^{-\frac{\|\mathbf{w}\|^2}{2}}}{(2\pi)^{d/2}}$ is the density of the standard Gaussian distribution $\mathcal{N}(0, I)$ from which the weights are drawn, so that the integral is exactly the expectation of the product of the two units' activations over all possible weight vectors $\mathbf{w}$.

In other words, the kernel $k_n(\mathbf{x}, \mathbf{y})$ evaluates the *exact* inner product of the feature representations produced by an infinitely wide single-layer network with the activation g_n , without ever instantiating that infinite layer. The integer n is the *degree* of the kernel and selects the activation function g_n applied by the hidden units: for instance, $n = 0$ yields $g_0(z) = \Theta(z)$, a threshold activation that outputs 1 for positive inputs and 0 otherwise (the unit simply fires or stays silent), while $n = 1$ yields $g_1(z) = \Theta(z) z = \max(0, z)$, which is mathematically identical to the Rectified Linear Unit (ReLU) activation introduced in the previous sections. Each choice of n thus corresponds to a different infinitely wide neural layer, and Cho and Saul [10] further show that these kernels can be composed in multiple layers, mimicking the behaviour of a deep network.

While the exact computation of the arc-cosine kernel provides a powerful theoretical link between SVMs and ANNs, computing the full kernel matrix remains computationally intractable for large datasets, as discussed in Section 2.2.2.

3 State Of The Art

The integration of kernel methods and ANNs has been pursued from several distinct directions, each combining the two paradigms through a different mechanism. This chapter organises the relevant literature into five strands. We begin with the theoretical correspondence that links infinitely wide networks to kernels (Section 3.1), which provides the formal basis for the entire field and supports the arc-cosine kernel introduced in Section 2.4. We then review the two dominant constructive approaches: using a neural network as a feature extractor for a kernel machine (Section 3.2), and stacking kernel machines into explicitly deep architectures (Section 3.3), the latter being the line of work that this thesis extends. We subsequently discuss deep architectures built without backpropagation (Section 3.4), which motivate the training philosophy adopted here, and recent kernel machines that learn features directly (Section 3.5). Finally, Section 3.6 positions the present work with respect to this landscape.

3.1 The Neural-Kernel Correspondence

The deepest connection between the two paradigms is theoretical: under certain limits, neural networks and kernel methods are formally equivalent. This observation is the foundation on which the rest of the field is built.

The starting point of this strand is the work of Cho and Saul [9], who introduced the arc-cosine kernels already described in Section 2.4. By showing that the inner product of the features computed by an infinitely wide single-layer network with random Gaussian weights admits a closed-form kernel expression, they provided an explicit bridge between a neural layer and a kernel, and further showed that such kernels can be composed in multiple layers to emulate a deep network.

This was later generalised in two influential ways. The first is the *Neural Network Gaussian Process* (NNGP) view: as the width of a randomly initialised network tends to infinity, the function it represents converges to a Gaussian process whose covariance is the NNGP kernel, so that Bayesian inference with the network reduces to kernel regression [32, 25]. The second is the *Neural Tangent Kernel* (NTK) of Jacot et al. [20], which characterises not the network at initialisation but its *training dynamics*: in the infinite-width limit, gradient-descent training of a network behaves like kernel gradient descent with a fixed kernel, the NTK. Together, these results establish that under certain conditions, wide neural networks and kernel methods are two views of the same object. While these analyses are primarily theoretical and are not in themselves practical learning algorithms, they justify the central premise of this work: that a kernel built to imitate a neural layer is a principled building block.

3.2 Deep Kernel Learning: Networks as Feature Extractors

A first constructive way of combining the two paradigms is to place a deep neural network *in front of* a kernel machine, using the network to learn a rich, data-dependent representation and the kernel to perform the final prediction. The most prominent example is the *Deep Kernel Learning* (DKL) framework of Wilson et al. [44], in which the inputs of a base kernel are transformed by a deep architecture, and the parameters of the resulting deep kernel are learned jointly through the marginal likelihood of a Gaussian process. This yields a model that retains the non-parametric flexibility and uncertainty estimates of Gaussian processes while benefiting from the representational power of deep networks, and that scales to large datasets through inducing-point and structure-exploiting approximations.

This idea has been extended in several directions, including kernels with recurrent structure for sequential data [38] and latent-variable formulations that regularise the learned representation [27]. More recently, the *Deep Kernel Machines* (DKM) of Yang et al. and Milsom et al. [45, 31] revisit the infinite-width limit itself, modifying it so that intermediate-layer representations are no longer fixed but adapt to the data, and derive a practical deep learning algorithm from deep Gaussian processes, including a convolutional variant competitive with neural networks on image benchmarks. A common characteristic of this entire family is that the model is trained *end-to-end* by gradient-based optimisation of a single global objective, typically a (variational) marginal likelihood. This contrasts with the convex, layer-wise training pursued in this thesis.

3.3 Deep Architectures of Stacked Kernel Machines

The strand most directly related to this thesis builds depth not by feeding a network into a single kernel, but by *stacking kernel machines into multiple layers*, so that each layer transforms the representation produced by the previous one. This is the natural kernel-based analogue of a deep network, and it is the family that the present work belongs to and extends.

An early and influential idea was the analytic composition of arc-cosine kernels [9]: because each arc-cosine kernel corresponds to a neural layer, composing L of them yields a kernel that mimics an L -layer network. This gives rise to *multilayer kernel machines*, in which the kernel itself is deep. While elegant, this analytic composition does not scale to large datasets, since it still requires the full kernel matrix; subsequent work explored core vector machines as a scaling-up mechanism for deep arc-cosine support vector machines [3], addressing precisely the scalability problem that motivates the present thesis, though through a different route.

In parallel, several authors built deep models from SVMs as the base unit. Wiering et al. proposed the Multi-Layer Support Vector Machine (MLSVM) and the Neural Support Vector Machine [43, 42], deep SVM models trained by a gradient-

based rule on a min-max formulation of the optimisation problem, reporting improvements over a standard SVM on regression benchmarks. More recently, Wang et al. introduced the *SVM-based Deep Stacking Network* (SVM-DSN) [41], which organises linear SVMs in the Deep Stacking Network architecture of Deng and colleagues [13, 12] and introduces a backpropagation-like layered tuning scheme to optimise all base-SVMs jointly while preserving their individual convexity. Related constructions include the D-SVM of Abdullah et al. [1], which feeds the kernel activations of support vectors as inputs to the next layer and combines several such models in an ensemble, and the stacked-SVM feature extractor of Kim et al. [22], in which higher-layer SVMs were observed to generalise better than lower ones.

Of particular relevance to this thesis is the line of work by Mehrkanoon and Suykens on *deep hybrid neural-kernel networks* [30]. Their model uses an explicit feature map based on random Fourier features to make the transition between the neural and kernel components straightforward, and crucially to make the model scalable by solving the optimisation problem in the primal rather than forming the kernel matrix. Stacking several such hybrid layers yields a deep architecture, and follow-up work extended this with additional block types such as average, maxout, and convolutional kernel blocks. The use of an explicit random-feature map to obtain scalability and primal-form training is mechanically the closest precedent to the approach taken in this work; the present thesis differs in its use of the arc-cosine feature map and in a fully greedy, layer-wise training procedure.

Building directly upon these scalable, primal-form approximations, recent work by Acero and Belanche [2] proposed a greedy Multi-Layer Support Vector Machine (ML-SVM) in which linear SVMs are stacked across layers, each layer transforming its input through a Gaussian RBF kernel approximation based on random Fourier features and feeding the resulting SVM outputs to the next layer. The architecture is trained greedily, one layer at a time, solving only linear optimisation problems and avoiding both kernel matrices and backpropagation, and was shown to match a standard RBF SVM on large datasets while training substantially faster. A structural limitation of this design, however, is that only the low-dimensional SVM decision values are propagated between layers, a single value per class, so that the rich, high-dimensional representation built inside each layer is discarded before reaching the next. This thesis identifies this information bottleneck as a central obstacle to making depth consistently beneficial, and the architecture proposed here is explicitly designed to address it, using an arc-cosine rather than an RBF feature map.

3.4 Deep Learning Without Backpropagation

A separate but conceptually important strand questions whether depth requires backpropagation at all. The most prominent example is the *Deep Forest* (gcForest) of Zhou and Feng [47], which constructs a deep model from non-differentiable modules, namely cascaded layers of random forests. The authors conjecture that the

effectiveness of deep models stems from three characteristics, layer-by-layer processing, in-model feature transformation, and sufficient model complexity, none of which intrinsically requires differentiable modules or gradient-based end-to-end training. Each layer is trained independently and its outputs are concatenated with the original features and passed to the next layer, so that the architecture grows greedily without ever backpropagating an error signal. Related ideas include Forward Thinking, which builds deep networks one layer at a time by passing the outputs of each layer forward [19], and the original Deep Stacking Networks [13], whose blocks are trained layer by layer and which provided the architectural template later adopted by SVM-DSN.

This strand is relevant to the present work not because it uses kernels, but because it validates the design philosophy adopted here: a deep architecture can be built by stacking easy-to-train, individually optimal modules in a greedy, layer-wise fashion, retaining the layer-by-layer feature transformation that gives deep models their power while avoiding the non-convex optimisation and tuning difficulties of backpropagation.

3.5 Feature-Learning Kernel Machines

A final and more recent strand seeks to endow kernel machines with the feature-learning ability that was long considered the exclusive advantage of neural networks. Radhakrishnan et al. [33] introduced *Recursive Feature Machines* (RFMs), kernel machines that explicitly learn features by alternating between two steps: fitting a kernel predictor, and reweighting the input features using the *Average Gradient Outer Product* (AGOP) of that predictor, a statistical estimator that up-weights the directions most relevant to the output. Iterating this process allows a kernel machine to recursively refine its representation, and the authors posit, through the Deep Neural Feature Ansatz, that this same mechanism underlies feature learning in deep neural networks. Subsequent analysis by Gan and Poggio [15] showed that AGOP can be understood as a greedy approximation to gradient descent, which is of direct relevance to the greedy training strategy adopted in this thesis. A related convolutional construction is given by the Convolutional Kernel Networks of Mairal et al. [28], which approximate a hierarchical kernel through a network-like architecture of feature maps.

3.6 Positioning of This Work

The methods reviewed above show that the gap between kernel methods and neural networks has been bridged from several angles, yet no existing approach occupies the specific intersection targeted by this thesis. The Gaussian-process deep kernels of Section 3.2 and the recent deep kernel machines are trained end-to-end by gradient methods, sacrificing the convexity that makes SVMs attractive. Among the stacked-

SVM models of Section 3.3, the multilayer arc-cosine kernel relies on analytic kernel composition that does not scale, while SVM-DSN and the Neural Support Vector Machine reintroduce a backpropagation-like global tuning step that forfeits the simplicity of purely greedy training. The backpropagation-free architectures of Section 3.4 are not kernel-based, and the feature-learning kernels of Section 3.5 operate on a single shallow kernel rather than a deep stack.

The closest precedent is the greedy ML-SVM of Acero and Belanche [2], which is already convex, greedy, and scalable. It differs from the present work in two essential respects. First, it propagates only the low-dimensional SVM decision values between layers, creating the information bottleneck discussed in Section 3.3. Second, it relies on a Gaussian RBF kernel approximation.

This thesis aims to fill that gap by developing a deep architecture that is simultaneously built upon the convex, margin-based formulation of support vector machines; scalable to large datasets; and trained in a fully greedy, layer-wise manner that avoids backpropagation entirely. The remainder of this document details this architecture and evaluates each of its design decisions empirically.

4 My Proposal

This chapter introduces the proposed architecture, PROPOSAL- NAME TO DEFINE (PROPOSAL), a deep, convex, and backpropagation-free model that stacks support vector machines into multiple layers. Section 4.1 describes the architecture and the principle that distinguishes it from its predecessor. Section 4.2 specifies the arc-cosine feature map that gives each layer its interpretation as an infinite-width ANN layer. Section 4.3 presents the greedy layer-wise learning algorithm, Section 4.4 the prediction procedure, and Section 4.5 the analysis of computational complexity. Finally, Section 4.6 shows that the architecture is in fact kernel-agnostic, the arc-cosine map being one principled choice among others.

4.1 Architecture

4.1.1 Motivation: Avoiding Representation Collapse

The point of departure for this work is the greedy multilayer SVM (ML-SVM) of Acero and Belanche [2], described in Section 3.3. As established there, that architecture propagates only the scalar SVM decision value between consecutive layers. Since the decision value lies in R for regression, and in R^K for a K -class problem, the high-dimensional representation built inside each layer through the random-feature map is discarded before reaching the next layer. We refer to this loss of information as *representation collapse*: the inter-layer signal is constrained to a dimension fixed by the task rather than by the model’s capacity, so that adding layers contributes little, as the experiments of Acero and Belanche on large datasets confirm.

The architecture proposed here, PROPOSAL, resolves this limitation by a single structural change: instead of forwarding the scalar decision values, it forwards a *learned linear projection of the full feature representation*. The dimension of the inter-layer representation then becomes a free parameter of the model, independent of the number of classes, and representation collapse becomes structurally impossible.

4.1.2 Layer Structure

Let $\mathbf{X}^{(0)} = \mathbf{X} \in R^{n \times d}$ denote the input data, with n samples and d features. The architecture consists of L stacked blocks. Each block ℓ applies two successive transformations to its input $\mathbf{X}^{(\ell)}$:

1. A *non-linear* transformation through a random-feature map, $\mathbf{X}^{(\ell)} \mapsto \Phi^{(\ell)} \in R^{n \times P}$, where P is the number of random features. The map is parameterised by a weight matrix $\Omega^{(\ell)}$ sampled according to the chosen kernel (Section 4.2).

2. A *linear* transformation, $\Phi^{(\ell)} \mapsto \Phi^{(\ell)} \mathbf{W}^{(\ell)} = \mathbf{X}^{(\ell+1)}$, learned by a block of m parallel linear SVMs.

Each of the m SVMs in block ℓ is trained on the same feature map $\Phi^{(\ell)}$ and produces a weight vector $\mathbf{w}_k^{(\ell)} \in R^P$. These are collected column-wise into the projection matrix

$$\mathbf{W}^{(\ell)} = [\mathbf{w}_1^{(\ell)}, \dots, \mathbf{w}_m^{(\ell)}] \in R^{P \times m}, \quad (59)$$

so that the inter-layer representation $\mathbf{X}^{(\ell+1)} = \Phi^{(\ell)} \mathbf{W}^{(\ell)} \in R^{n \times m}$ has dimension m . The design intent is $P \gg m$: the block expands the data into a high-dimensional feature space and then compresses it into a learned m -dimensional subspace, retaining far more information than a single scalar decision value. After the final block, a single linear SVM is trained on $\mathbf{X}^{(L)}$ to produce the model's output.

This structure mirrors a neural-network layer, a non-linear map followed by a learned linear projection, with one essential difference: each linear map $\mathbf{W}^{(\ell)}$ is the solution of m convex optimisation problems, rather than the result of gradient descent. There is no backpropagation, and the margin-based guarantees of the SVM are preserved at every layer.

The use of m parallel SVMs per block, rather than one, provides the projection matrix $\mathbf{W}^{(\ell)}$ with m columns and thus controls the dimension of the inter-layer representation. How these m SVMs are made to differ, for instance by assigning them distinct regularisation constants, is a design choice that influences the diversity of the learned projection; its effect is studied empirically in Chapter 5 rather than assumed here.

4.2 The Arc-Cosine Feature Map

The defining choice of the proposed architecture is the feature map used in each block. We employ the arc-cosine kernel of Cho and Saul [9], introduced in Section 2.4, because it endows each block with a precise interpretation as an infinite-width ANN layer. Recall that the n -th order arc-cosine kernel is defined directly as an expectation over Gaussian weights $\mathbf{w} \sim \mathcal{N}(0, I)$:

$$k_n(\mathbf{x}, \mathbf{z}) = E_{\mathbf{w} \sim \mathcal{N}(0, I)} [\phi_{\mathbf{w}}^{(n)}(\mathbf{x}) \phi_{\mathbf{w}}^{(n)}(\mathbf{z})], \quad \phi_{\mathbf{w}}^{(n)}(\mathbf{x}) = \sqrt{2} \Theta(\mathbf{w}^\top \mathbf{x}) (\mathbf{w}^\top \mathbf{x})^n. \quad (60)$$

4.2.1 The Random Feature Map

Because the kernel is already written as an expectation, it admits an immediate Monte Carlo approximation, as described in general terms in Section 2.2.2. Drawing

$\omega_j \sim \mathcal{N}(0, I)$ independently for $j = 1, \dots, P$, the feature map is

$$\Phi_j^{(\ell)}(\mathbf{x}) = \sqrt{\frac{2}{P}} \max(0, \omega_j^\top \mathbf{x}^{(\ell)})^n. \quad (61)$$

The inner product $\Phi^{(\ell)}(\mathbf{x})^\top \Phi^{(\ell)}(\mathbf{z})$ is then an unbiased estimator of $k_n(\mathbf{x}, \mathbf{z})$, convergent as $P \rightarrow \infty$ by the law of large numbers. This construction is a direct application of Monte Carlo integration to an expectation that Cho and Saul already identify; no phase variable is required, since the features are real and non-negative by construction.

4.2.2 Why Bochner’s Theorem Is Not Needed

The random Fourier features commonly used to approximate the RBF kernel are justified by Bochner’s theorem, which applies only to shift-invariant kernels. Arc-cosine kernels are not shift-invariant: $k_n(\mathbf{x}, \mathbf{z})$ depends on the norms of $\mathbf{x}$ and $\mathbf{z}$ and the angle between them, not on their difference, so Bochner’s theorem does not apply. This raises three questions, each resolved without it. *Positive-definiteness* follows directly from the integral representation: any function expressible as $E[\phi_{\mathbf{w}}(\mathbf{x})\phi_{\mathbf{w}}(\mathbf{z})]$ is a valid covariance and hence positive-definite. *Unbiasedness* of the estimator follows from the same expectation form by the law of large numbers. The *sampling distribution* is not a free choice to be fixed by a theorem, as it is for shift-invariant kernels, but is prescribed directly as $\mathcal{N}(0, I)$ by the integral representation. Consequently, the arc-cosine map requires no bandwidth parameter: the scale of the kernel is determined entirely by the data norms, and since the data is standardised before training, $\mathcal{N}(0, I)$ is both the theoretically prescribed and the practically sufficient choice. This is a simplification relative to standard random Fourier features.

4.2.3 Theoretical Interpretation and Its Scope

Using arc-cosine features of degree $n = 1$, for which the activation is $\max(0, u)$, the ReLU, gives each block a precise meaning: the non-linear map $\Phi^{(\ell)}$ is a finite-sample approximation to the arc-cosine kernel of degree 1, which is exactly the kernel induced by an infinite-width single-layer ReLU network with $\mathcal{N}(0, I)$ weights. The projection matrix $\mathbf{W}^{(\ell)}$ is the learned component, playing the role of the output weights of that infinite network, determined by a convex SVM solve rather than by backpropagation. This interpretation is the reason the arc-cosine map is adopted as the definitive form of the architecture: it realises, in a convex and backpropagation-free model, a finite-sample analogue of a deep ReLU network.

The scope of this claim must be stated precisely. The equivalence holds *per block*: each block individually approximates one arc-cosine kernel layer. It does not extend to the full stacked architecture. The composed kernel of Cho and Saul is defined by a

recursion that uses exact kernel values at each layer, whereas PROPOSAL forwards a learned linear projection $\mathbf{X}^{(\ell+1)} = \Phi^{(\ell)} \mathbf{W}^{(\ell)}$ whose values are determined by the training labels, not by the kernel recursion. The stacked architecture therefore does not approximate the L -fold composed arc-cosine kernel, and no such claim is made.

4.2.4 Practical Considerations

Two practical points follow from this choice. First, the architecture requires no bandwidth parameter, as argued above. Second, the features $\max(0, \omega^\top \mathbf{x})^n$ are unbounded and grow with $\|\mathbf{x}\|$; consequently, the inter-layer representation $\mathbf{X}^{(\ell+1)}$ must be standardised before the next feature map to prevent scale drift across layers, using statistics fixed at training time. Standard scaling between blocks is sufficient.

4.3 Learning Algorithm

The architecture is trained greedily, one block at a time. Each block is trained to completion before the next is constructed, using the output of the previous block as its input. Because each block reduces to the training of independent linear SVMs, each of which is a convex problem, the procedure inherits the optimisation guarantees of the SVM and requires no global, end-to-end optimisation. Algorithm 1 summarises the procedure.

Algorithm 1 PROPOSAL training

Require: Training data $(\mathbf{X}, \mathbf{y})$; number of layers L ; SVMs per block m ; regularisations $C_1, \dots, C_m$; random features P

Ensure: Per-block models $\{\mathbf{W}^{(\ell)}, \Omega^{(\ell)}\}_{\ell=0}^{L-1}$, final SVM $\mathbf{w}^{(L)}$

```

1:  $\mathbf{X}^{(0)} \leftarrow \mathbf{X}$ 
2: for  $\ell = 0, \dots, L - 1$  do
3:   Sample  $\Omega^{(\ell)} \sim \mathcal{N}(0, I)$  ▷ arc-cosine weights
4:    $\Phi^{(\ell)} \leftarrow \phi(\Omega^{(\ell)}, \mathbf{X}^{(\ell)}) \in R^{n \times P}$  ▷ feature map, Eq. 61
5:   for  $k = 1, \dots, m$  do
6:      $\mathbf{w}_k^{(\ell)} \leftarrow \text{LINEARSVM}(\Phi^{(\ell)}, \mathbf{y}; C_k)$ 
7:   end for
8:    $\mathbf{W}^{(\ell)} \leftarrow [\mathbf{w}_1^{(\ell)}, \dots, \mathbf{w}_m^{(\ell)}]$ 
9:    $\mathbf{X}^{(\ell+1)} \leftarrow \Phi^{(\ell)} \mathbf{W}^{(\ell)}$  ▷ forward the projection, not the decision values
10: end for
11:  $\mathbf{w}^{(L)} \leftarrow \text{LINEARSVM}(\mathbf{X}^{(L)}, \mathbf{y}; C)$ 

```

The weight matrices $\Omega^{(\ell)}$ sampled for the feature maps must be stored, since they are required to reproduce the same transformations at prediction time. Each linear SVM is solved in its primal form (Section 4.5), which is the source of the architecture’s scalability.

4.4 Prediction

Prediction applies the stored transformations of each block in sequence. Given a new input $\mathbf{x}$, the representation is propagated forward through the L blocks: at each block ℓ , the stored sampling matrix $\mathbf{\Omega}^{(\ell)}$ defines the feature map $\mathbf{\Phi}^{(\ell)}$, and the stored projection $\mathbf{W}^{(\ell)}$ produces the next representation $\mathbf{x}^{(\ell+1)} = \mathbf{\Phi}^{(\ell)}(\mathbf{x}) \mathbf{W}^{(\ell)}$, which is standardised using the training-time statistics before the next block. The final linear SVM is then applied to $\mathbf{x}^{(L)}$ to produce the prediction.

4.5 Computational Complexity

The scalability of PROPOSAL rests on a single principle: the Gram matrix is never formed. We develop the argument here, as it is central to the architecture’s applicability in the large- n regime.

An exact kernel SVM requires the $n \times n$ kernel matrix, at a cost of $O(n^2d)$ to build and between $O(n^2)$ and $O(n^3)$ to solve (Section 2.2.2); both costs grow at least quadratically in the number of samples n , which is precisely what renders exact kernel SVMs intractable for large datasets. PROPOSAL replaces this with an explicit random-feature map. By construction, the random-feature matrix $\mathbf{\Phi}^{(\ell)} \in R^{n \times P}$ satisfies $\mathbf{\Phi}^{(\ell)} \mathbf{\Phi}^{(\ell)\top} \approx \mathbf{K}$, so a linear SVM trained on the rows of $\mathbf{\Phi}^{(\ell)}$ approximates a kernel SVM without ever materialising $\mathbf{K}$.

Concretely, each linear SVM is solved in its primal form,

$$\min_{\mathbf{w} \in R^P} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n L\left(y_i, \mathbf{w}^\top \mathbf{\Phi}_i^{(\ell)}\right), \quad (62)$$

which operates directly on the rows of $\mathbf{\Phi}^{(\ell)}$ and never forms the product $\mathbf{\Phi}^{(\ell)} \mathbf{\Phi}^{(\ell)\top}$. The primal and dual solutions are equivalent by strong duality, but only the dual requires the Gram matrix; solving in the primal is what avoids the quadratic cost.

The choice of the primal is deliberate and tied to the regime the architecture targets. The primal variable $\mathbf{w}$ lies in R^P , so the cost of the primal solve scales with the number of random features P ; the dual variable $\boldsymbol{\alpha}$ lies in R^n and depends on the data through the $n \times n$ Gram matrix $\mathbf{\Phi}^{(\ell)} \mathbf{\Phi}^{(\ell)\top}$. The primal is therefore the more efficient formulation precisely when $n \gg P$, and the dual when $P \gg n$. Since PROPOSAL targets the large-sample regime, in which n may reach the order of millions while P is a fixed hyperparameter of moderate size, the condition $n \gg P$ holds by design, and the primal is the appropriate choice. Crucially, solving the dual would reintroduce the $n \times n$ Gram matrix that the random-feature approximation was introduced to avoid, reinstating the quadratic memory and super-quadratic time cost of the exact kernel SVM and thereby negating the entire computational benefit of the approach. The random-feature map and the primal solve are thus two halves

of a single design: once the kernel is made explicit through the feature map $\Phi^{(\ell)}$, the kernel trick, whose natural home is the dual, is no longer required, and the problem is most naturally and most cheaply solved as a linear model trained directly on the explicit features. In the opposite regime $P > n$ the dual would be preferable, but that regime lies outside the scope of this work, whose aim is to remain tractable as n grows large.

The cost of a single block is therefore the sum of three linear-in- n terms: constructing $\Phi^{(\ell)}$ costs $O(nPd_\ell)$, where d_ℓ is the input dimension of the block ($d_0 = d$, and $d_\ell = m$ for $\ell \geq 1$); training the m linear SVMs costs, for each, time linear in n for a fixed P , since the solver operates by repeated passes over the n rows of $\Phi^{(\ell)}$ at a per-pass cost of $O(nP)$; and forming the projection $\Phi^{(\ell)}\mathbf{W}^{(\ell)}$ costs $O(nPm)$. Summing over L blocks, the total training cost is linear in n for fixed P , m , and L . This is the decisive contrast with the exact kernel SVM: PROPOSAL trades the quadratic-to-cubic dependence on n for a linear one, at the price of approximating the kernel through P random features, which makes the architecture tractable in the large- n regime where exact SVMs cannot be applied.

4.6 Generality of the Architecture

The arc-cosine map of Section 4.2 is the definitive instantiation of the proposal, but it is not intrinsic to it. The architecture, a stack of blocks, each pairing a non-linear feature map with a learned linear SVM projection, is in fact *kernel-agnostic*: any feature map may be substituted, provided the dimensional structure of the projection is respected. We make this generality precise, since it clarifies which part of the proposal is the contribution (the architecture) and which is a principled choice within it (the arc-cosine kernel).

The general requirement is simply that each block produce an inter-layer representation $\mathbf{X}^{(\ell+1)} \in R^{n \times m}$, in which each of the m SVMs of the block contributes one column. Two cases arise. If all m SVMs in a block share a single feature map $\Phi^{(\ell)} \in R^{n \times P}$, their weight vectors are conformable, and the block reduces to the compact matrix form $\mathbf{X}^{(\ell+1)} = \Phi^{(\ell)}\mathbf{W}^{(\ell)}$ of Equation 59; this is the case used throughout this thesis. If instead each SVM k uses its own feature map $\Phi_k^{(\ell)} \in R^{n \times P_k}$, the weight vectors have differing dimensions and cannot be stacked into a single matrix; the block is then defined by the per-SVM projections $\mathbf{X}_{\cdot k}^{(\ell+1)} = \Phi_k^{(\ell)}\mathbf{w}_k^{(\ell)}$, concatenated column-wise. In either case the inter-layer representation has the same shape $n \times m$, and the next block proceeds identically.

This freedom permits several variants. The most natural alternative is the *RBF variant*, in which each block uses the random Fourier feature map of the Gaussian kernel [34], with a per-layer bandwidth estimated by the median heuristic; its cosine features are bounded in $[-1, 1]$, which removes the need for inter-layer standardisation. This variant lacks the infinite-width ReLU interpretation of the arc-cosine map, but is architecturally identical, and serves in Chapter 5 as a control that isolates the

effect of the kernel choice from that of the architecture. More generally, different kernels may be used at different layers, or even at different SVMs within a layer, allowing the representation to be refined by heterogeneous transformations. We do not pursue these heterogeneous configurations empirically; we note them to make clear that the proposed contribution is the stacking architecture and its projection mechanism, of which the arc-cosine kernel is the theoretically motivated, but not the only possible, realisation.

5 Experiments

6 Conclusion

References

- [1] Azizi Abdullah, Remco C Veltkamp, and Marco A Wiering. “An ensemble of deep support vector machines for image categorization”. In: *2009 International Conference of Soft Computing and Pattern Recognition*. IEEE. 2009, pp. 301–306.
- [2] Joan Acero-Pousa and Lluís A Belanche-Muñoz. “A new approach to multilayer SVMs”. In: *ESANN 2025 proceedings, European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning*. 2025, pp. 467–472.
- [3] AL Afzal and S Asharaf. “Deep multiple multilayer kernel learning in core vector machines”. In: *Expert Systems with Applications* 96 (2018), pp. 149–156.
- [4] Shun-ichi Amari. “Backpropagation and stochastic gradient descent method”. In: *Neurocomputing* 5.4-5 (1993), pp. 185–196.
- [5] Christopher M Bishop. *Pattern recognition and machine learning*. Springer, 2006.
- [6] Salomon Bochner. *Vorlesungen über Fouriersche Integrale*. Akademische Verlagsgesellschaft m.b.H., 1932.
- [7] Bernhard E Boser, Isabelle M Guyon, and Vladimir N Vapnik. “A training algorithm for optimal margin classifiers”. In: *Proceedings of the fifth annual workshop on Computational learning theory*. 1992, pp. 144–152.
- [8] Jair Cervantes et al. “A comprehensive survey on support vector machine classification: Applications, challenges and trends”. In: *Neurocomputing* 408 (2020), pp. 189–215.
- [9] Youngmin Cho and Lawrence Saul. “Kernel methods for deep learning”. In: *Advances in neural information processing systems* 22 (2009).
- [10] Youngmin Cho and Lawrence K Saul. “Large-margin classification in infinite neural networks”. In: *Neural computation* 22.10 (2010), pp. 2678–2697.
- [11] Corinna Cortes and Vladimir Vapnik. “Support-vector networks”. In: *Machine learning* 20.3 (1995), pp. 273–297.
- [12] Li Deng, Xiaodong He, and Jianfeng Gao. “Deep stacking networks for information retrieval”. In: *2013 IEEE International Conference on Acoustics, Speech and Signal Processing*. IEEE. 2013, pp. 3153–3157.
- [13] Li Deng and Dong Yu. “Deep convex net: A scalable architecture for speech pattern classification”. In: *Proceedings of the Interspeech*. 2011, pp. 2285–2288.
- [14] Manuel Fernández-Delgado et al. “Do we need hundreds of classifiers to solve real world classification problems?” In: *The journal of machine learning research* 15.1 (2014), pp. 3133–3181.

- [15] Yulu Gan and Tomaso Poggio. *For hyperbfs agop is a greedy approximation to gradient descent*. Tech. rep. Center for Brains, Minds and Machines (CBMM), 2024.
- [16] Ian Goodfellow et al. *Deep learning*. Vol. 1. 2. MIT press Cambridge, 2016.
- [17] Werner H Greub. *Linear algebra*. Vol. 23. Springer Science & Business Media, 2012.
- [18] Maarten Grootendorst. “The Kernel Trick”. In: *TowardsDataScience* (2018). Accessed: June 8, 2024. URL: <https://towardsdatascience.com/the-kernel-trick-c98cdbcaeb3f>.
- [19] Chris Hettinger et al. “Forward thinking: Building and training neural networks one layer at a time”. In: *arXiv preprint arXiv:1706.02480* (2017).
- [20] Arthur Jacot, Franck Gabriel, and Clément Hongler. “Neural tangent kernel: Convergence and generalization in neural networks”. In: *Advances in neural information processing systems* 31 (2018).
- [21] William Karush. “Minima of functions of several variables with inequalities as side constraints”. In: *M. Sc. Dissertation. Dept. of Mathematics, Univ. of Chicago* (1939).
- [22] Sangwook Kim, Swathi Kavuri, and Minhoo Lee. “Deep Network with Support Vector Machines”. In: *Neural Information Processing*. Ed. by Minhoo Lee et al. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 458–465. ISBN: 978-3-642-42054-2.
- [23] Joseph-Louis Lagrange. “Méthode des multiplicateurs de Lagrange”. In: *Journal de l’École Polytechnique* 13 (1811), pp. 141–175.
- [24] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. “Deep learning”. In: *nature* 521.7553 (2015), pp. 436–444.
- [25] Jaehoon Lee et al. “Deep neural networks as gaussian processes”. In: *arXiv preprint arXiv:1711.00165* (2017).
- [26] Jiyeon Lee. *Support Vector Regression*. <https://leejiyeon52.github.io/Support-Vector-Regression/>. Accessed: 2024-06-12. 2018.
- [27] Haitao Liu et al. “Deep latent-variable kernel learning”. In: *IEEE Transactions on Cybernetics* 52.10 (2021), pp. 10276–10289.
- [28] Julien Mairal et al. “Convolutional kernel networks”. In: *Advances in neural information processing systems* 27 (2014).
- [29] Warren S McCulloch and Walter Pitts. “A logical calculus of the ideas immanent in nervous activity”. In: *The bulletin of mathematical biophysics* 5 (1943), pp. 115–133.
- [30] Siamak Mehrkanoon and Johan AK Suykens. “Deep hybrid neural-kernel networks using random Fourier features”. In: *Neurocomputing* 298 (2018), pp. 46–54.

- [31] Edward Milsom, Ben Anson, and Laurence Aitchison. “Convolutional deep kernel machines”. In: *arXiv preprint arXiv:2309.09814* (2023).
- [32] Radford M Neal. *Bayesian learning for neural networks*. Vol. 118. Springer Science & Business Media, 2012.
- [33] Adityanarayanan Radhakrishnan et al. “Mechanism for feature learning in neural networks and backpropagation-free machine learning models”. In: *Science* 383.6690 (2024), pp. 1461–1467.
- [34] Ali Rahimi and Benjamin Recht. “Random features for large-scale kernel machines”. In: *Advances in neural information processing systems* 20 (2007).
- [35] Frank Rosenblatt. “The perceptron: a probabilistic model for information storage and organization in the brain.” In: *Psychological review* 65.6 (1958), p. 386.
- [36] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. “Learning representations by back-propagating errors”. In: *nature* 323.6088 (1986), pp. 533–536.
- [37] Sagar Sharma, Simone Sharma, and Anidhya Athaiya. “Activation functions in neural networks”. In: *Towards Data Sci* 6.12 (2017), pp. 310–316.
- [38] Maruan Al-Shedivat et al. “Learning scalable deep kernels with recurrent structure”. In: *Journal of Machine Learning Research* 18.82 (2017), pp. 1–37.
- [39] Vladimir Vapnik and Alexey Chervonenkis. “Theory of pattern recognition”. In: (1974).
- [40] Analytics Vidhya. *Insight into SVM (Support Vector Machine) along with code*. <https://www.analyticsvidhya.com/blog/2021/04/insight-into-svm-support-vector-machine-along-with-code/>. Apr. 2021.
- [41] Jingyuan Wang, Kai Feng, and Junjie Wu. “SVM-based deep stacking networks”. In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 33. 01. 2019, pp. 5273–5280.
- [42] Marco A Wiering and Lambert RB Schomaker. “Multi-layer support vector machines”. In: *Regularization, optimization, kernels, and support vector machines* 19 (2014), p. 457.
- [43] Marco A Wiering et al. “The neural support vector machine”. In: *BNAIC 2013: Proceedings of the 25th Benelux Conference on Artificial Intelligence, Delft, The Netherlands, November 7-8, 2013*. Delft University of Technology (TU Delft); under the auspices of the Benelux ... 2013.
- [44] Andrew Gordon Wilson et al. “Deep kernel learning”. In: *Artificial intelligence and statistics*. PMLR. 2016, pp. 370–378.
- [45] Adam X Yang et al. “A theory of representation learning gives a deep generalisation of kernel methods”. In: *International Conference on Machine Learning*. PMLR. 2023, pp. 39380–39415.

- [46] Nicholas Young. *An introduction to Hilbert space*. Cambridge university press, 1988.
- [47] Zhi-Hua Zhou and Ji Feng. “Deep forest”. In: *National science review* 6.1 (2019), pp. 74–86.